BridgeIR: An Agent-Operated, Runtime-Extensible Multi-Backend IR Stack

Hocky Yudhiono
(draft — authors TBD)

October 13, 2026

Abstract

Draft. BridgeIR is a multi-backend intermediate-representation stack whose extension surface — pipelines, pass definitions, verifiers, and invariants — is expressed as data that an automated agent can compose and load at runtime. We adopt MLIR as the IR substrate and contribute the layers it lacks: a tiered extension model with precise rebuild boundaries, a runtime control plane, and first-class invariants that make agent-authored transformations trustworthy. **TODO:** sharpen thesis, state contributions, results.

1 Introduction

Compiler pipelines are increasingly built and operated by automated agents, yet the frameworks they must extend were designed for humans editing C++ and waiting for rebuilds. Adding a pass, an operation definition, a verifier, or an invariant to an MLIR-based stack today means editing TableGen and C++ sources, rebuilding the tool, and restarting the process. That authoring loop — not the IR data structure — is the bottleneck.

Our thesis is that the IR *core* is not the right place to compete: MLIR has won the core, and systems that try to replace it lose the ecosystem while chasing a moving target. What is missing around the core are (i) declarative extension tiers that an agent can author as data, (ii) a control plane that discovers, validates, and loads those extensions at runtime, and (iii) first-class invariants that make agent-authored transformations trustworthy.

Contributions (draft).

- A tiered extension model for agent-authored compiler extensions — from pipeline manifests and interpreted transformations up to runtime operation definitions and compiled plugins — with precise rebuild boundaries. **TODO**
- A runtime control plane: a manifest registry, plugin loader, and structured (machine-readable) diagnostics. **TODO**
- Invariants as data: per-stage contracts checked at runtime. **TODO:** decide SMT-checked vs. translation-validation depth.
- An evaluation protocol for *agent operability*: task success, time and cost to first successful extension, error recovery, versus a vanilla out-of-tree MLIR workflow. **TODO:** define tasks, baselines, metrics.

Axis	MLIR today	Where "bigger" lives
Multi-level structure	Top form: a metalanguage; multi-level IRs are cheap	— (no rival supersedes it)
Declarative definitions (ops/types/attrs)	Top form: ODS/TableGen generates builders, accessors, verifiers	—
Interpreted transformations	Young: PDL patterns and transform-dialect scripts	E-graph rewriting: egg [14], egglog [16]; tensor superoptimization: TASO [4]
Rewrite search and scheduling	Greedy, order-sensitive; fixpoints via ad-hoc pass orders	Equality saturation; relational e-matching [15]; rule inference [7]
Per-compilation specialization	Static pipelines; passes fixed at build time; schedules left to backends	Autotuned schedules (TVM [1], Triton [13]); peephole synthesis (STOKE [11]); generative compilation (SPIRAL [9]); staged specialization (Futamura [2])
Semantics and guarantees	Local per-op C++ verifiers; no formal semantics	Translation validation (Alive2) [6]; mechanized semantics (Vellvm) [17]; SMT-backed contracts
Incrementality	Whole-pass recomputation	Demand-driven compilation: Adapton [3], Salsa [12]
Runtime extensibility	Second-class: `ExtensibleDialect` with C++ verify callbacks; `.so` plugins with exact-ABI requirements	Data-first extension registries; sandboxed plugin hosts (WASM)
Agent operability	Weak: extension means editing `.td`/C++ and rebuilding	Plan → build → load loops; machine-readable pass/dialect descriptions; structured diagnostics
Stability and interchange	No core stability; dialect-level fixes (StableHLO) [8]	Versioned dialects with serialization discipline
Schedule and hardware semantics	Partial: affine, linalg, CIRCT	Schedule-first IRs: Halide [10], TVM TensorIR [18], ISL schedule trees

Table 1: Axis-by-axis comparison. "Bigger" names the direction that pushes an axis past MLIR, not a single competing system.

2 Positioning: Where MLIR Ends

We frame the design space as a set of axes for MLIR [5]. For each axis we ask what is available today and where systems that go *beyond* it live. "Bigger" here does not mean a single system that dominates MLIR — no such system exists today — but rather the research or engineering direction that pushes that particular axis past what MLIR offers. Table 1 states the comparison.

Reading the table: the first two rows are the foundation we inherit unchanged. The middle rows constitute the design space BridgeIR occupies — interpreted transformations, trustworthy rewriting, per-compilation specialization, invariants, incrementality, runtime extensibility, and agent operability. The last rows we adopt as discipline (versioned dialects, stable serialization) and per-backend work (schedule and hardware semantics). **TODO:** add a per-axis "BridgeIR stance" once the design section stabilizes.

References

[1] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. TVM: An automated end-to-end optimizing compiler for deep learning. In *Proceedings of the USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2018.

[2] Yoshihiko Futamura. Partial evaluation of computation process — an approach to a compiler-compiler. *Systems, Computers, Controls*, 2(5):45–50, 1971.

[3] Matthew A. Hammer, Khoo Yit Phang, Michael Hicks, and Jeffrey S. Foster. Adapton: Composable demand-driven incremental computation. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2014.

[4] Zhihao Jia, Oded Padon, James Thomas, Todd Warszawski, Matei Zaharia, and Alex Aiken. TASO: Optimizing deep learning computation with automatic generation of graph substitutions. In *Proceedings of the ACM Symposium on Operating Systems Principles (SOSP)*, 2019.

[5] Chris Lattner, Mehdi Amini, Uday Bondhugula, Albert Cohen, Andy Davis, Jacques Pienaar, River Riddle, Tatiana Shpeisman, Nicolas Vasilache, and Oleksandr Zinenko. MLIR: Scaling compiler infrastructure for domain specific computation. In *Proceedings of the IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, 2021.

[6] Nuno P. Lopes, Juneyoung Lee, Chung-Kil Hur, Zhengyang Liu, and John Regehr. Alive2: Bounded translation validation for LLVM. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2021.

[7] Chandrakana Nandi, Max Willsey, Amy Zhu, Yisu Remy Wang, Brett Saiki, Adam Anderson, Adriana Schulz, Dan Grossman, and Zachary Tatlock. Rewrite rule inference using equality saturation. In *Proceedings of the ACM on Programming Languages (OOPSLA)*, 2021.

[8] OpenXLA Community. StableHLO: A stable HLO dialect. `https://github.com/openxla/stablehlo`. Accessed 2026.

[9] Markus Püschel, José M. F. Moura, Jeremy R. Johnson, David Padua, Manuela M. Veloso, Bryan Singer, Jianxin Xiong, Franz Franchetti, Aca Gačić, Yevgen Voronenko, Kang Chen, Robert W. Johnson, and Nicholas Rizzolo. SPIRAL: Code generation for DSP transforms. *Proceedings of the IEEE*, 93(2):232–275, 2005.

[10] Jonathan Ragan-Kelley, Connelly Barnes, Andrew Adams, Sylvain Paris, Frédo Durand, and Saman Amarasinghe. Halide: A language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2013.

[11] Eric Schkufza, Rahul Sharma, and Alex Aiken. Stochastic superoptimization. In *Proceedings of the ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2013.

[12] The Rust Project. Salsa: Incremental compilation for Rust. `https://github.com/salsa-rs/salsa`. Accessed 2026.

[13] Philippe Tillet, H. T. Kung, and David Cox. Triton: An intermediate language and compiler for tiled neural network computations. In *Proceedings of the ACM SIGPLAN International Workshop on Machine Learning and Programming Languages (MAPL)*, 2019.

[14] Max Willsey, Chandrakana Nandi, Yisu Remy Wang, Oliver Flatt, Zachary Tatlock, and Pavel Panchekha. egg: Fast and extensible equality saturation. In *Proceedings of the ACM on Programming Languages (POPL)*, volume 5, 2021.

[15] Yihong Zhang, Yisu Remy Wang, Max Willsey, and Zachary Tatlock. Relational e-matching. In *Proceedings of the ACM on Programming Languages (POPL)*, 2022.

[16] Yihong Zhang, Yisu Remy Wang, Oliver Flatt, David Cao, Philip Zucker, Eli Rosenthal, Zachary Tatlock, and Max Willsey. Better together: Unifying datalog and equality saturation. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2023.

[17] Jianzhou Zhao, Santosh Nagarakatte, Milo M. K. Martin, and Steve Zdancewic. Formalizing the LLVM intermediate representation for verified program transformations. In *Proceedings of the ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, 2012.

[18] Lianmin Zheng, Ruochen Liu, Junru Shao, Tianqi Chen, Joseph E. Gonzalez, Ion Stoica, and Ameer Haj-Ali. TensorIR: An abstraction for automatic tensorized program optimization. In *Proceedings of the ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2023.